TP 06 — Descente de gradient

Inside ESILV : une journée dans la peau d'un élève-ingénieur

Sixtine Michel

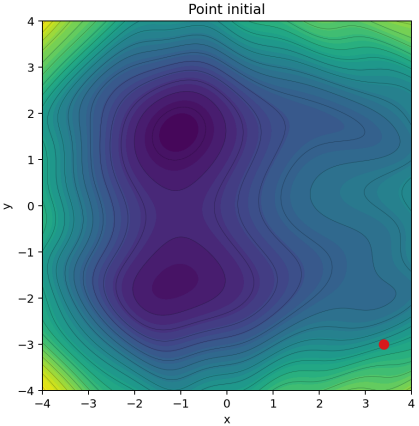
De Vinci Higher Education — ESILV

Notre mission

Objectif

Faire descendre une bille sur un paysage mathématique jusqu'à un minimum local.

À chaque étape, elle mesure la pente puis avance dans le sens opposé.



D'un point de vue algorithmique

Le calcul est partagé entre quatre fonctions simples :

```
def fonction(x, y):
    # Donner l'altitude au point (x, y).
def gradient_numerique(x, y):
    # Estimer la pente selon x et selon y.
def effectuer_pas(point, taux_apprentissage):
    # Avancer dans la direction opposee au gradient.
def calculer_trajectoire(point_initial, nb_iterations,
                         taux_apprentissage):
    # Repeter les pas et memoriser le parcours.
```

La visualisation affiche ensuite la trajectoire obtenue sur le relief.

La fonction à minimiser

$$f(x, y) = 0{,}18(x^2 + y^2) + 0{,}8 \sin x + 0{,}6 \cos(1{,}5y) + 0{,}15 \sin(xy).$$

Elle associe une altitude à chaque point (x, y). Les oscillations créent plusieurs reliefs et rendent la trajectoire visible.

Que représente le gradient ?

$$\nabla f(x, y) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right).$$

Le gradient pointe dans la direction où la fonction augmente le plus vite. Pour descendre, on suit donc $-\nabla f$.

Une dérivée numérique

$$\frac{\partial f}{\partial x}(x, y) \approx \frac{f(x+\varepsilon, y) - f(x-\varepsilon, y)}{2\varepsilon}.$$

Cette différence centrée compare deux valeurs placées symétriquement. On applique la même idée selon y.

La règle de descente

$$P_{k+1} = P_k - \eta \nabla f(P_k).$$

η est le taux d'apprentissage : trop petit, la descente est lente ; trop grand, elle peut osciller ou diverger.

Les cinq étapes du programme

1. définir la fonction ;
2. approximer son gradient ;
3. effectuer un pas de descente ;
4. mémoriser toute la trajectoire ;
5. préparer les tableaux de visualisation.

Correspondance entre étapes et TODO

1. Fonction mathématique	**TODO 1**
2. Gradient numérique	**TODO 2, 3 et 4**
3. Pas de descente	**TODO 5 et 6**
4. Trajectoire	**TODO 7, 8 et 9**
5. Grille et valeurs affichées	**TODO 10, 11 et 12**

Évaluer la fonction — TODO 1

À vous

- Evaluer les quatre termes de la fonction.

Évaluer la fonction — TODO 1

```
return (0.18*(x**2 + y**2)
        + 0.8*np.sin(x)
        + 0.6*np.cos(1.5*y)
        + 0.15*np.sin(x*y))
```

Les opérations NumPy fonctionnent pour un nombre comme pour un tableau.

Dérivée selon x — TODO 2

À vous

- Approximer df/dx par difference centree.

Dérivée selon x — TODO 2

```
dfdx = (fonction(x + epsilon, y)
        - fonction(x - epsilon, y)) / (2*epsilon)
```

Dérivée selon y — TODO 3 et 4

À vous

- Approximer df/dy par difference centree.
- Renvoyer [dfdx, dfdy] dans un tableau de flottants.

Dérivée selon y — TODO 3 et 4

```
dfdy = (fonction(x, y + epsilon)
        - fonction(x, y - epsilon)) / (2*epsilon)
return np.array([dfdx, dfdy], dtype=float)
```

Effectuer un pas — TODO 5 et 6

À vous

- Calculer le gradient aux coordonnees du point courant.
- Calculer point - taux_apprentissage*gradient.

Effectuer un pas — TODO 5 et 6

```
1 gradient = gradient_numerique(point[0], point[1])
2 nouveau_point = point - taux_apprentissage*gradient
```

Le signe moins

Avec un signe plus, la bille monterait vers les valeurs élevées.

Initialiser la trajectoire — TODO 7

À vous

- Initialiser la trajectoire et le point avec des copies du point initial.

Initialiser la trajectoire — TODO 7

```
trajectoire = [point_initial.copy()]
point = point_initial.copy()
```

`copy()` évite que les éléments mémorisés changent avec le point courant.

Itérer — TODO 8 et 9

À vous

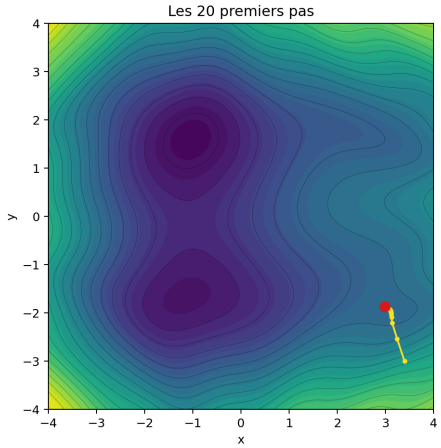
- Effectuer un nouveau pas et mettre à jour le point courant.
- Ajouter une copie du nouveau point a trajectoire.

Itérer — TODO 8 et 9

```python
for _ in range(nb_iterations):
    point = effectuer_pas(point, taux_apprentissage)
    trajectoire.append(point.copy())
return np.array(trajectoire)
```

Pour N pas, le tableau contient $N + 1$ points, avec le départ.

Les premiers pas



Au début, le gradient est grand et les déplacements sont visibles. Ils diminuent près du minimum.

Créer le paysage — TODO 10 et 11

À vous

- Creer 250 valeurs regulieres sur chaque axe, puis construire X et Y.
- Evaluer fonction(X, Y) sur toute la grille.

Créer le paysage — TODO 10 et 11

```
xs = np.linspace(X_MIN, X_MAX, 250)
ys = np.linspace(Y_MIN, Y_MAX, 250)
X, Y = np.meshgrid(xs, ys)
Z = fonction(X, Y)
```

`X`, `Y` et `Z` ont tous la forme $(250, 250)$.

Suivre le coût — TODO 12

À vous

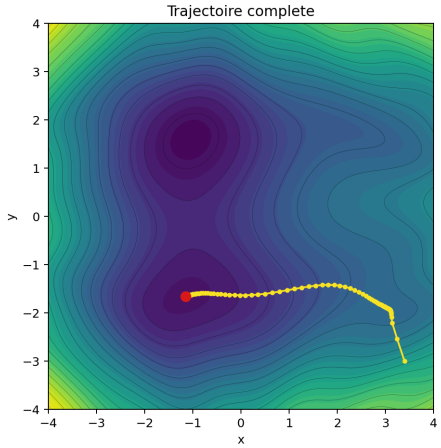
- Evaluer la fonction sur les deux colonnes de trajectoire.

Suivre le coût — TODO 12

```
valeurs = fonction(trajectoire[:, 0],
                   trajectoire[:, 1])
```

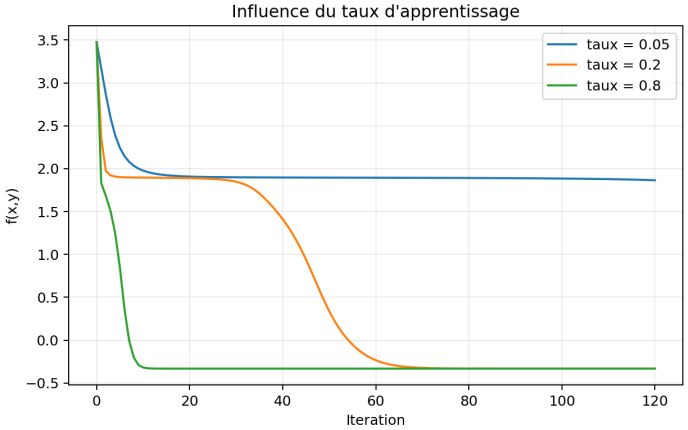
La courbe doit décroître : chaque point est plus bas que le précédent.

Le minimum atteint



Après 120 pas, la norme du gradient est inférieure à 3×10^{-4} : la bille est pratiquement immobile au fond de la vallée.

Choisir le taux d'apprentissage



Expérimenter

- essayer $\eta = 0{,}05$, 0,2 puis 0,8 ;
- déplacer `POINT_INITIAL` ;
- modifier `epsilon` ;
- augmenter `NB_ITERATIONS`.

Question

Le même point initial conduit-il toujours au même minimum ? Et deux départs différents ?

Lancer le TP dans Spyder

1. Ouvrir `06-descente_de_gradient.py`.
2. Choisir le backend **Automatic** ou **Qt**.
3. Redémarrer la console après un changement de backend.
4. Compléter les TODO dans l'ordre.

Animation dynamique

`plt.show(block=True)` garde la fenêtre ouverte.

Concepts appris

- fonction de deux variables ;
- dérivée numérique ;
- gradient ;
- norme ;
- boucle itérative ;
- taux d'apprentissage ;
- copie de tableau ;
- `meshgrid` et vectorisation.

À vous de coder !